

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 13 · LECTURE NOTES

Shipping Geometric Code

What to write, what to use, and what to promise.

MODULE OUTCOMES

- Choose a robustness strategy from the requirements.
- Separate predicates from constructions in an interface.
- Test geometric code in ways that actually find bugs.
- Judge when to use a library and which one.
- State what geometric code can honestly promise.

1 Choosing a strategy

If your code...	Then use...	Because
only classifies existing points — no new coordinates are computed	Exact filtered predicates (Module 02).	Roughly 2× the cost of naive, and always correct. This row covers more than people expect: convex hulls, triangulation, Delaunay, point location, range searching, and visibility all construct nothing.
constructs points, within a bounded domain	Fixed-point integer coordinates, scaled onto a grid.	Inputs become exactly representable and comparisons become trivially reliable. The most underrated option (Module 06 §4), and what Clipper does.
chains constructions — the output of one operation feeds the next	Snap rounding.	Stops error accumulating across the chain, at the cost of possible topology change. Document the displacement bound.
must produce provably exact answers	An exact rational kernel (CGAL).	Correct, and slow, with coordinate size growing through the computation. CAD and verification need it; real-time does not.
is interactive and can tolerate approximation	Filtered predicates with a tolerant topological layer.	Fast, and the limits must be documented rather than discovered.

SEPARATE PREDICATES FROM CONSTRUCTIONS IN THE INTERFACE

Predicates can be made exact affordably — Module 02 established that. **Constructions cannot be made exact at all in fixed precision**, because the result is simply not representable (Module 06 §4). These are different problems with different costs, and conflating them is the most common design error in geometric libraries.

So make the distinction visible in the API. CGAL parameterises its kernels on exactly this axis, and `Exact_predicates_inexact_constructions_kernel` is the most-used type in the library — a name that is both unwieldy and exactly right, because it states the guarantee.

Then a caller knows what it is getting: either a correct *combinatorial* answer with coordinates rounded to double, or both exact at a substantial cost, and the choice is theirs to make with information.

And most callers want the first. The topology must be right — the hull must be convex, the triangulation must cover, the mesh must be watertight — **while the coordinates can be a few ULPs off without anyone noticing or caring.** That is the cheapest useful guarantee in geometric computing, and recognising that it is usually sufficient is worth a great deal.

2 Testing

- **Property-based tests rather than example-based ones.** The hull is convex and contains every input point; the triangulation has exactly $n - 2$ triangles and covers the polygon without overlap; the area of a union is at most the sum of the areas. **These hold for every input**, so they can be checked against generated data rather than hand-written cases.
- **Degenerate generators** from Module 01 §4 — lattices, collinear runs, duplicates, one-ULP pairs, cocircular sets, axis-aligned configurations. **These are the tests; random points are not.**
- **Exact rational ground truth** on small inputs. Too slow to ship and ideal for deciding what the answer should have been.
- **Metamorphic tests:** translate, rotate, scale, reflect, and permute the input; the output must transform correspondingly or be unchanged. **No expected output is needed**, which is what makes them cheap to write.
- **Differential testing** against CGAL or another implementation on the same inputs, which catches disagreements neither side's own tests would find.
- **And render every failure.** A failed geometric test is nearly unreadable as numbers and obvious as a picture — which is why Module 01 asked for SVG output on day one.

PERMUTING THE INPUT IS THE BEST SINGLE TEST

The output of a geometric algorithm should not depend on the order in which the input was supplied — up to a documented tie-breaking rule for genuine ties. A convex hull is a property of a point set, not of a list.

So shuffle the input and re-run. If the answer changes, you have found either a degeneracy resolved inconsistently or a predicate that is not exact — and both are real bugs that will surface on real data.

It requires no ground truth, no reference implementation, and no expected output — only two runs of your own code that must agree with each other. That makes it the cheapest useful test in the whole subject.

And it catches precisely the bugs that matter, because an inconsistent predicate is order-sensitive almost by definition (Module 01 §2): the same three points evaluated in different orders gave different answers, and a permuted input is exactly how that reaches the surface. **If you add one test to a geometry codebase, add this one.**

3 Libraries

Library	What it covers	Character
CGAL	Essentially everything in this course, and a great deal more.	Exact, correct, comprehensive, and heavy. Deep template machinery and long compile times. GPL or commercial licence , which has decided a great many architectural choices in industry.
Clipper2	2D boolean operations and polygon offsetting.	Integer coordinates, genuinely robust, small and readable. Boost licence. The best demonstration of Module 06 §4's integer strategy.
Triangle (Shewchuk)	2D constrained Delaunay and quality meshing.	The reference implementation for Module 09 §3. Free for non-commercial use — check before shipping.
earcut	Polygon triangulation with holes.	Tiny, fast, permissively licensed, and readable in an afternoon. Used in map rendering at scale.
qhull	Convex hulls, Delaunay, and Voronoi in arbitrary dimension.	The lifting map of Module 08 §3 applied. Mature and widely wrapped — it is what SciPy calls.
libigl, Open3D, MeshLab	Mesh processing, repair, and analysis (Module 12 §4).	Research-oriented with good defaults. Excellent for pipelines and for one-off repair work.

WHEN TO WRITE IT YOURSELF

When the problem is small and the dependency is not worth it. A convex hull is twenty lines (Module 03 §1); pulling in CGAL for it is not a sensible trade.

When you need a specific degeneracy policy the library does not offer. The collinear-points decision of Module 03 §1 is the clearest example — if you need every boundary point and the library returns only extreme ones, no amount of wrapping fixes it.

When the data layout matters. If the library's point type forces a conversion inside a hot loop, the conversion can easily cost more than the algorithm (CSCE 735 Module 02).

And to understand it. Which is the point of this course — you cannot evaluate a library's guarantees, read its kernel documentation, or diagnose its failure on your data without knowing what the guarantees could be and where they come from. **Writing it once is what makes using someone else's an informed decision rather than a hopeful one.**

4 Claiming honestly

- **'Predicates are exact; constructions are rounded to double.'** Precise, honest, and it tells the caller exactly what they have — correct topology, approximate coordinates.

- **'Degeneracies are resolved by symbolic perturbation with index-based tie-breaking'** — which says that the output is a valid structure for an infinitesimally nearby input, and warns the caller who actually wanted to detect the degeneracy (Module 02 §3).
- **'Output coordinates are snap-rounded to a grid of size g , and no point is displaced by more than d .'** A quantified promise rather than a vague one.
- **'Tested against exact rational arithmetic for $n \leq 100$, differentially against CGAL above that, and under permutation at every size.'** States the evidence, not the hope.
- **And what you cannot honestly say: 'it is robust'. 'Robust' is the word in this field that means nothing,** and it is the one most frequently used. Say which inputs are handled, which guarantees hold, and which cases are known to fail — a documented limitation is engineering, and an undocumented one is a bug waiting for someone else to find.

WHERE THIS LEAVES YOU

You can implement the standard structures — convex hulls, triangulations, planar subdivisions, Voronoi and Delaunay diagrams, spatial indices — **and for each one you know which predicate it rests on and which degeneracies attack it.**

And you know the thing most people working with geometry do not: that geometric failures are overwhelmingly about arithmetic rather than about algorithms, that the distinction between a predicate and a construction determines what is possible, and that both are fixable by known means at known cost.

CSCE 629 supplied the complexity analysis. CSCE 641 and 645 supplied the representations. CSCE 649 assumed the collision geometry worked, and CSCE 647 assumed the acceleration structures did.

This course built what those assumed. And the two courses beside it in this semester build the rest of what an engine needs that the degree did not cover — **CSCE 605 the compiler that turns shaders and scripts into something the machine runs, and CSCE 678 the network that lets more than one person be in the world at once.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CGAL — kernel documentation and the robustness manual (free)	The §1 strategy table, as a working library's design. The kernel-choice discussion is the best practical writing on this subject.
Kettner et al. — Classroom Examples of Robustness Problems (free)	Worth re-reading now that you can fix everything in it.
Chen, Cheung, Xie, Zhang & Zhang — Metamorphic testing: a review (free)	The §2 technique, generalised well beyond geometry.

Resource	What it covers for this module
Shewchuk — Lecture notes on geometric robustness (free)	The practitioner's summary, from the person who did most to make this tractable.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take each algorithm you built in this course and classify it as predicate-only or construction-involving.
- 2 **Design an interface** that makes the predicate/construction distinction explicit, and implement two kernels behind it.
- 3 Convert one construction-heavy algorithm to integer coordinates and measure the robustness difference.
- 4 **Write property-based tests** for your hull, triangulation, and boolean implementations.
- 5 **Add the permutation test** to every algorithm you have written. Report what it finds — it will find something.
- 6 Add metamorphic tests for translation, rotation, and scaling.
- 7 Build a differential test harness against CGAL and run it on your degenerate generator.
- 8 Write the documentation paragraph for one of your routines, meeting §4's standard.
- 9 Find a geometry library's documentation that says 'robust' and determine what it actually guarantees.
- 10 **Project 2 is now due.** Submit the implementation, the timing study across input distributions, the library comparison, the degeneracy statement, and the rendered figures.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five situations and the robustness strategy each calls for.
- 2 Why separate predicates from constructions in an interface?
- 3 What is the cheapest useful guarantee, and why is it usually enough?
- 4 Name six ways to test geometric code.
- 5 Why is permuting the input the best single test?
- 6 Name six libraries and what each covers.
- 7 Give four reasons to write geometry yourself.
- 8 Give four honest claims and one thing you cannot say.